

GitHub Copilot under Token-Based Billing

A Practical Guide

Bernardo Cohen

June 2026

Purpose. GitHub replaced Copilot’s premium-request system with a token-based one on 1 June 2026. A chat turn no longer has a fixed cost: it now scales with the model you pick, the context you attach, the conversation history that replays each turn, and (in Agent mode) every tool output the loop has accumulated. The cost spread between best-case and worst-case use of the same feature is one to two orders of magnitude. This guide is the single document to read end-to-end so the office can budget the new pricing without losing the productivity gains Copilot already delivers — and so we can defend the budget without retreating from the tool.

Contents

1	What changed on 1 June 2026	2
2	Why this matters	2
3	Where the money actually goes — the five surfaces	2
4	The cost equation — a mental model	3
5	The four levers, in priority order	3
6	The two-lane workflow	4
7	Worked examples	4
8	Anti-patterns — stop doing this	5
9	Agent-mode brief template	5
10	Quick-wins checklist	6
11	What you are trading	6
12	The five rules	7
A	Order-of-magnitude pricing (per 1M tokens, USD)	7
B	Sources	7

1 What changed on 1 June 2026

The unit moved from *premium requests* (count of turns $\times$ per-model multiplier, fixed monthly quota) to *AI Credits* (denominated in dollars; 1 credit = \$0.01), billed by tokens consumed times a per-model rate. The same surfaces are paid (chat, edit, agent, code review) and the same surfaces are free (inline completion, Next Edit Suggestions).

	Old system	New system
Unit	“Premium request”	AI Credit (1 credit = \$0.01)
What you spend	Counts of requests	Tokens $\times$ per-model price
Plan allowance	N requests / month	\$ in credits / month
Paid surfaces	Chat, edits, agent	Same — + code review
Still free	Code completion, NES	Code completion, NES
Annual subscribers	—	Stay on legacy multipliers until renewal

Table 1: Billing-model migration at a glance.

Monthly allowance per plan (per user, no rollover):

Plan	Monthly price	Monthly AI Credits
Copilot Pro (individual)	\$10	\$10
Copilot Pro+ (individual)	\$39	\$39
Copilot Business (per seat)	\$19	\$19
Copilot Enterprise (per seat)	\$39	\$39

Overage is purchasable on paid plans, subject to admin policy — confirm with whoever owns the org contract whether overage spending is enabled or capped.

2 Why this matters

A realistic Agent-mode session on a premium model spends **\$0.50 to \$5+** of credits. Public reports document developers burning **\$30–\$40 in a single morning** on long agentic sessions that recursed through tests and fixes. Pro is \$10 per month. The arithmetic is unforgiving if Copilot is used casually.

The good news. The highest-frequency Copilot interactions — inline completion and Next Edit Suggestions — remain **unmetered**. Most of the productivity we get from Copilot lives in the free lane. The optimisations in this document are about preserving that lane’s value while spending dramatically less on the paid lane.

The thesis. Copilot’s per-turn cost is now a function of choices the user makes (which model, what context, how long the chat). Once those choices are deliberate rather than reflexive, *individual* bills typically drop 50–80% with no loss of output quality — and often with a gain, because the same discipline that produces cheaper turns produces better-shaped prompts.

3 Where the money actually goes — the five surfaces

Copilot in VS Code is five distinct surfaces with very different cost behaviour. *Knowing which surface you are on is the single biggest determinant of how fast your credits drain.*

Agent mode is where budgets vanish. Each tool call’s output (a file you asked it to read, a command result, a test log) gets re-injected into every *subsequent* turn’s input. A 20-tool-call

Surface	Trigger	Bills?	Tokens per turn	Cost shape
Code completion	Just type — ghost text	No	n/a	Free, unlimited
Next Edit Suggestions	Tab through proposed edits	No	n/a	Free, unlimited
Chat — Ask mode	Side panel, plain question	Yes	~3–15K in, ~0.5–2K out	Small per turn
Chat — Edit mode	Side panel, “Edits” target	Yes	~10–40K in, ~1–4K out	Medium; $\propto$ files edited
Chat — Agent mode	Side panel, “Agent” + tools	Yes	50K–500K cumulative	Highest — loops + tool re-injection
Code Review (PR/inline)	“Copilot review” on a PR	Yes	Depends on diff size	Per review, on top of chat

Table 2: The five surfaces. Free, unlimited inline completion and NES are where most of Copilot’s daily value lives — use them aggressively.

session on a premium model routinely accumulates 200K+ input tokens, which is what drove the publicly-reported “\$30–40 in one morning” incidents.

4 The cost equation — a mental model

On any paid surface, the per-turn cost decomposes as

$$\text{cost} = \underbrace{(\text{prompt} + \text{context} + \text{history} + \text{tool outputs})}_{\text{input tokens}} \cdot r_{\text{in}} + \underbrace{(\text{reply} + \text{tool-call args})}_{\text{output tokens}} \cdot r_{\text{out}}.$$

Two facts worth internalising:

- **Output tokens are 5–6× more expensive than input** on most models. A whole-file rewrite (Edit mode) is dominated by its output.
- **Cached input is 10× cheaper than fresh input** on every model. Re-attaching the same file within a few minutes is nearly free; restarting a chat throws that cache away.

Back-of-envelope conversions (memorise one of these):

- *Sonnet 4.6, normal chat turn*: ~5K in + ~1K out = 3 credits $\Rightarrow$ Pro’s \$10 buys $\approx$ 330 such turns/month.
- *Opus, heavy refactor turn*: ~30K in + ~3K out = 22.5 credits $\Rightarrow$ Pro’s \$10 buys $\approx$ 44 turns.
- *Agent session, 20 tool calls on Opus*: ~200K cumulative in + ~15K out = 137.5 credits $\Rightarrow$ **one session $\approx$ 14% of Pro’s monthly pool.**

5 The four levers, in priority order

1. **Model choice.** A 10–40× spread between the cheapest and the most premium models. For routine work the *quality* gap is usually negligible.
2. **Tool-output history** (agent mode only — but dominant there). Shorter sessions, fresh chats per task, don’t re-purpose a 30-turn agent context.

Tier	Examples	When to use
Cheap	Haiku 4.5, GPT-5 mini, Gemini 3 Flash, Raptor mini	Exploration, “what does this do”, summarising, planning, simple edits
Standard	Sonnet 4.6, GPT-5.4, Gemini 3.1 Pro, MAI-Code-1-Flash	Multi-file edits, careful implementation, code review of a tricky diff
Premium	Opus 4.7 / 4.8, GPT-5.5, large-context GPT-5.4 / Gemini 3.1 Pro	Hard reasoning, debugging across a large codebase, architectural design

Table 3: Model tiers and where each earns its keep. Action: set the default chat model in VS Code to a cheap-tier model; escalate deliberately.

3. Attached context size. Cheapest → most expensive:

typed code in prompt < #selection < #editor < #file:name < @workspace /
#codebase < dragged folder

Dragging a folder into chat can be 10–100× what you actually needed.

4. **Chat history length.** Every prior turn replays as input on the next turn. A 20-turn chat at 5K context/turn sends 100K input tokens on turn 21 — one expensive turn. New chat per topic change.

6 The two-lane workflow

The strongest single habit you can build:

```

+-----+
EXPLORE ---> | Cheap lane (Haiku, mini) | <-- ask, plan, summarise
+-----+
              | plan locked
              v
+-----+
EXECUTE ---> | Premium lane (Sonnet+) | <-- one well-formed impl turn
+-----+
              |
              v
+-----+
VERIFY ---> | Cheap lane again | <-- review diff, write tests
+-----+

```

Framing. Premium models execute *locked* decisions; cheap models explore *undecided* ones. Reversing this is the most common way to burn the budget.

7 Worked examples

Prompt-chipping vs. one good brief

Anti-pattern — four turns, four bills, each carrying the entire prior history:

```

Turn 1: "Refactor this to use a dict."
Turn 2: "Actually use defaultdict."
Turn 3: "Make the keys strings."
Turn 4: "Add type hints."

```

Pattern — one complete brief, one turn, one bill:

```
"Refactor this to use a collections.defaultdict[str, list[Foo]],
add type hints, and keep the existing public signature."
```

The discipline that produces the cheaper version is the same discipline that produces better output. The two reinforce each other.

Reusable context lives in files, not chats

If you find yourself re-pasting “remember, we follow X convention” into every chat, move it into a file Copilot auto-loads. At the repo root, create `.github/copilot-instructions.md`:

```
- Python 3.12, type hints required on all public functions.
- No bare 'except'. Use specific exception types.
- SQLAlchemy 2.x -- no Query API; use select().
- Tests live in 'tests/', one file per source module.
- Plotting through internal pricebook.viz only -- never raw matplotlib.
```

Copilot loads this on every chat in that repo. The tokens are included in every turn’s input but are typically eligible for cached-input pricing (10× cheaper than fresh input), and you stop re-explaining the same conventions across chats. For repeated task shapes, drop `.github/prompts/<name>.prompt.md` files for things like “write unit tests for the selected function in our test style” and invoke them as one-liners.

8 Anti-patterns — stop doing this

- **✗ Asking yes/no on Opus.** 15–25K of context in plus a few hundred output tokens for a one-word answer is 10–15 credits. Do these on a cheap-lane model — or just look at the code.
- **✗ Dragging folders into chat.** You almost never want every file in the folder. Use `#file:` or `@workspace`.
- **✗ Prompt-chipping:** “refactor this” → “actually use defaultdict” → “now add type hints”. Four turns, four full bills. Write one complete brief.
- **✗ Re-running PR review on every push.** A Copilot review re-pays the whole diff each time. One review per meaningful change; human review for small follow-ups.
- **✗ Reusing a 30-turn agent session for a new task.** Old tool outputs are now in every new turn’s input.
- **✗ Defaulting to Agent mode** for things Ask could do. Agent mode is high-stakes, not chat-with-tools.

9 Agent-mode brief template

A well-shaped brief routinely cuts agent sessions from 20 turns to 4–6. Open with everything the agent needs to know, in one prompt:

```
Task: <one sentence>.
In scope: <files / packages>.
Out of scope: <what NOT to touch>.
Constraints: <patterns to preserve, libs required / forbidden>.
```

Definition of <how the agent knows to stop --- tests pass?
Done: endpoint responds? file exists?>.
At the end: <summarise files touched + assumptions made>.

Concrete example.

Task: Add /health endpoint to user-service returning
 {status:"ok", db:"ok"|"error"}.
In scope: services/user/handlers/, services/user/tests/.
Out of scope: any other service, infra config, CI.
Constraints: use existing FastAPI patterns; no new dependencies.
Definition of endpoint responds correctly, new test_health.py
 Done: passes locally.
At the end: list files touched + any assumptions made.

10 Quick-wins checklist

All of these cost nothing and all save credits.

- ✓ Set the default chat model to **Haiku 4.5** or **GPT-5 mini** in VS Code settings.
- ✓ Create `.github/copilot-instructions.md` in every active repo with your team conventions (auto-loaded, cache-eligible).
- ✓ Bind a keyboard shortcut to **New chat** so starting fresh is one keystroke.
- ✓ Glance at the **token counter** (bottom-right of the chat panel) before any turn that already has >30K context.
- ✓ Stop dragging folders into chat. Always `#file:` or `@workspace`.

Watch the counter — thresholds.

- Under ~30K tokens: comfortable.
- 30–80K: getting expensive on premium models; consider cutting losses if circling.
- 80K+: very expensive per turn. Stop, summarise, start fresh.

11 What you are trading

Each of the habits above feels like upfront work. The trade is **upfront thinking** against **downstream burn**.

Habit	Cost
Writing a complete prompt instead of chipping at it	+1 min thinking
Switching models mid-task	+5 sec of clicks
Starting a new chat per topic	+2 sec
Front-loading an agent brief	+1–2 min thinking
Effect on credit burn	–50% to –80% typical
Effect on output quality	Usually meaningfully better
Effect on rework	Lower

If a habit doesn't improve quality, it's still saving money. If it does both, it's a free upgrade.

12 The five rules

If you internalise only five things from this guide, internalise these.

1. **Free is free.** Code completion and Next Edit Suggestions don't bill. Default to them whenever the task is shaped like "complete this code".
2. **Cheap model first.** Explore on Haiku 4.5 or GPT-5 mini; escalate to a premium model only to execute a locked plan.
3. **Attach less.** `#selection` before `#editor` before `#file:` before `@workspace`. Never drag a folder.
4. **One task = one chat.** New chat per topic change. Long-running chats inflate every subsequent turn's input.
5. **Agent mode is high-stakes mode.** Front-load the brief, set a Definition of Done, and don't re-purpose a stale agent session.

A Order-of-magnitude pricing (per 1M tokens, USD)

Verified June 2026 against the GitHub Copilot [models-and-pricing reference page](#). Numbers move — re-check before procurement decisions.

OpenAI

Model	Input	Cached input	Output
GPT-5 mini	\$0.25	\$0.025	\$2.00
GPT-5.3-Codex	\$1.75	\$0.175	\$14.00
GPT-5.4 ($\leq 272K$ ctx)	\$2.50	\$0.25	\$15.00
GPT-5.4 ($> 272K$ ctx)	\$5.00	\$0.50	\$22.50
GPT-5.4 mini	\$0.75	\$0.075	\$4.50
GPT-5.4 nano	\$0.20	\$0.02	\$1.25
GPT-5.5 ($\leq 272K$ ctx)	\$5.00	\$0.50	\$30.00
GPT-5.5 ($> 272K$ ctx)	\$10.00	\$1.00	\$45.00

Anthropic

Model	Input	Cached input	Cache write	Output
Claude Haiku 4.5	\$1.00	\$0.10	\$1.25	\$5.00
Claude Sonnet 4 / 4.5 / 4.6	\$3.00	\$0.30	\$3.75	\$15.00
Claude Opus 4.5 / 4.6 / 4.7 / 4.8	\$5.00	\$0.50	\$6.25	\$25.00

Google

Other

B Sources

Model	Input	Cached input	Output
Gemini 2.5 Pro	\$1.25	\$0.125	\$10.00
Gemini 3 Flash	\$0.50	\$0.05	\$3.00
Gemini 3.1 Pro ($\leq 200K$ ctx)	\$2.00	\$0.20	\$12.00
Gemini 3.1 Pro ($> 200K$ ctx)	\$4.00	\$0.40	\$18.00
Gemini 3.5 Flash	\$1.50	\$0.15	\$9.00

Model	Input	Cached input	Output
Raptor mini	\$0.25	\$0.025	\$2.00
MAI-Code-1-Flash	\$0.75	\$0.075	\$4.50

Primary (official GitHub) — authoritative

- [Models and pricing for GitHub Copilot](#) — the rate card.
- [GitHub Blog — Copilot is moving to usage-based billing](#) — effective date, allowances, what stays free.
- [Plans for GitHub Copilot](#) — plan pricing.
- [VS Code Copilot FAQ](#).

Legacy (annual subscribers)

- [Model multipliers for annual plans \(legacy\)](#).
- [Overview of request-based billing \(legacy\)](#).

Community & practitioner

- [GitHub Community #163104 — Optimising Copilot Premium Request usage](#).
- [SmartScope — Premium request optimization: 8 anti-patterns](#).
- [A. Franceschelli — The “ask tool” trick](#).
- [GitHub Blog — Changes to Copilot Individual plans](#).

Industry coverage (context, not authoritative)

TheRouter.ai, ChatForest, DEV Community, prodSens, AI Tool Briefing, TechJournal, Medium reports — all documented user-reported cost spikes (10–50 $\times$ in extremes, \$30–40 agentic sessions).

What was NOT verified

- The retrieval mechanism behind `@workspace/#codebase` is not fully documented; the cost characterisation in Section 3 reflects observable behaviour, not a published spec.
- Per-surface tool-call output handling in Agent mode (full re-injection vs. summarisation) is implementation-defined. The “tool output stacks” claim is the right *budgeting* mental model regardless.

Last verified: 2026-06-08. When GitHub publishes a new rate card, re-fetch the models-and-pricing reference page and update Section A.